

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Отчёт по домашней работе № 2

по дисциплине «Расширенный курс по бэкенд-разработке в среде Node.js»

**Проектирование и технический дизайн API сервиса
«Lecter's Recipes»**

Вариант 5

Выполнил: студент группы БР1.1
Кошкарев Кирилл Павлович

Проверил:

Санкт-Петербург, 2026

1. Цель работы

Сформулировать функциональные и нефункциональные требования к бэкенд-приложению, выбрать формат реализации API, спроектировать все эндпоинты в виде спецификации OpenAPI с учётом схемы базы данных из ДЗ1 и описать тела запросов, ответов и возможные ошибки.

2. Функциональные требования

Таблица 1 — Функциональные требования и соответствующие эндпоинты

Код	Требование	Эндпоинты
FR-1	Регистрация пользователя с проверкой уникальности email и username	POST /auth/register
FR-2	Вход по email и паролю с выдачей пары JWT-токенов	POST /auth/login
FR-3	Продление сессии по refresh-токену без повторного ввода пароля	POST /auth/refresh
FR-4	Просмотр и редактирование собственного профиля	GET, PATCH /users/me
FR-5	Просмотр публичного профиля кулинара и его публикаций	GET /users/{id}, GET /users/{id}/recipes
FR-6	Поиск рецептов с фильтрацией по типу блюда, кухне, сложности, времени, ингредиентам и тегам	GET /recipes
FR-7	Сортировка выдачи по новизне, популярности и времени приготовления	GET /recipes?sort=
FR-8	Просмотр страницы рецепта: состав, шаги, медиа, автор	GET /recipes/{id}
FR-9	Создание, изменение и удаление собственного рецепта	POST, PATCH, DELETE /recipes
FR-10	Публикация черновика с проверкой готовности	POST /recipes/{id}/publish
FR-11	Управление составом, шагами и галереей рецепта	PUT /recipes/{id}/ingredients, /steps, POST /media
FR-12	Лайк рецепта и его снятие	POST, DELETE /recipes/{id}/like
FR-13	Сохранение рецепта в личный кабинет	POST, DELETE /recipes/{id}/favorite, GET /users/me/favorites
FR-14	Комментирование рецепта с ответами на комментарии	GET, POST /recipes/{id}/comments, PATCH, DELETE /comments/{id}
FR-15	Подписка на кулинара и лента его публикаций	POST, DELETE /users/{id}/follow, GET /users/me/feed
FR-16	Работа со справочниками категорий, кухонь, тегов и ингредиентов	GET, POST /categories, /cuisines, /tags, /ingredients

3. Нефункциональные требования

Таблица 2 — Нефункциональные требования

Код	Категория	Требование
NFR-1	Безопасность	Пароли хранятся только в виде bcrypt-хеши; поле password_hash исключено из всех ответов API
NFR-2	Безопасность	Доступ к изменяющим операциям — по JWT в заголовке Authorization; изменять рецепт может только автор или администратор
NFR-3	Безопасность	Все входные данные валидируются по схеме до обращения к базе; ответ 400 содержит перечень проблемных полей
NFR-4	Производительность	Списковые ответы всегда постраничные, размер страницы не более 100 записей
NFR-5	Производительность	Время ответа на запросы витрины — не более 300 мс на тестовом наборе данных
NFR-6	Совместимость	Версия API зафиксирована в URL (/api/v1); несовместимые изменения выпускаются новой версией
NFR-7	Предсказуемость	Единый формат ошибки для всех эндпоинтов: объект error с полями code, message и необязательным details
NFR-8	Документируемость	Спецификация OpenAPI публикуется приложением и доступна в Swagger UI по адресу /docs
NFR-9	Наблюдаемость	Эндпоинт /health возвращает состояние сервиса для проверок развёртывания
NFR-10	Переносимость	Конфигурация задаётся переменными окружения, секреты не хранятся в коде

4. Выбор формата API

Для проекта выбран **REST API**. Обоснование:

- предметная область хорошо раскладывается на ресурсы (рецепт, комментарий, подписка), а операции над ними естественно отображаются на методы HTTP;
- у сервиса один тип клиента — витрина рецептов, поэтому выделять отдельный слой Backend For Frontend под конкретный интерфейс преждевременно: это добавило бы промежуточный сервис без выигрыша;
- ресурсный API проще документировать в OpenAPI и тестировать в Postman, что требуется в ДЗЗ;
- кэширование ответов витрины средствами HTTP работает «из коробки» для GET-запросов.

Отдельные агрегирующие эндпоинты (GET /users/me/feed и GET /users/me/favorites), по сути решающие задачу BFF, реализованы как самостоятельные ресурсы — это избавляет клиента от серии запросов, не

нарушая ресурсной модели.

5. Соглашения проектирования

- базовый путь — `/api/v1`, формат обмена — `application/json; charset=utf-8`;
- имена ресурсов — существительные во множественном числе, вложенность не глубже двух уровней (`/recipes/{id}/comments`);
- действия, не выражаемые методом HTTP, оформлены как подресурс-команда: `POST /recipes/{id}/publish`;
- для частичного обновления используется `PATCH`, для полной замены коллекции — `PUT` (состав и шаги рецепта);
- включение и выключение реакции выражено парой `POST` и `DELETE` над одним ресурсом (`/recipes/{id}/like`);
- постраничность — параметры `page` (с единицы) и `limit` (1..100); ответ содержит объект `meta` с полями `page`, `limit`, `total`, `totalPages`;
- время передаётся строками ISO-8601 в UTC, идентификаторы — целые числа `int64`.

6. Коды ответов

Таблица 3 — Используемые коды ответов

Код	Когда возвращается	Пример ситуации
200 OK	Успешное чтение или изменение	GET /recipes, PATCH /users/me
201 Created	Создан новый ресурс	POST /auth/register, POST /recipes
204 No Content	Операция выполнена, тело не нужно	DELETE /recipes/{id}, POST /users/{id}/follow
400 VALIDATION_ERROR	Некорректное тело или параметры запроса	email неверного формата, limit=999
401 UNAUTHORIZED	Токен отсутствует, повреждён или истёк	GET /users/me без заголовка Authorization
401 INVALID_CREDENTIALS	Неверная пара email и пароль	POST /auth/login
403 FORBIDDEN	Пользователь аутентифицирован, но не имеет прав	Правка чужого рецепта или комментария
404 NOT_FOUND	Ресурс отсутствует или недоступен зрителю	Запрос чужого черновика
409 CONFLICT	Нарушение уникальности состояния	Повторная регистрация email, повторная подписка
422 UNPROCESSABLE	Данные корректны, но операция недопустима	Публикация рецепта без шагов

Код	Когда возвращается	Пример ситуации
500 INTERNAL_ERROR	Непредвиденная ошибка сервера	Сбой обращения к базе данных

7. Формат ошибки

Все ошибки возвращаются единой структурой, что позволяет клиенту обрабатывать их одним обработчиком, а поле `details` — подсвечивать конкретные поля формы.

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Проверьте корректность переданных данных",
    "details": [
      { "field": "email", "message": "Некорректный формат email" },
      { "field": "password", "message": "Пароль не короче 8 символов" }
    ]
  }
}
```

Листинг 1 — единый формат ответа с ошибкой

8. Спроектированные эндпоинты

Спецификация описывает 28 путей и 44 операции, сгруппированные по тегам. Полный текст — в файле `openapi.yaml`.

Таблица 4 — Перечень эндпоинтов API

Метод и путь	Назначение	Коды ответов
POST /auth/register	Регистрация	201, 400, 409
POST /auth/login	Вход	200, 400, 401
POST /auth/refresh	Обновление токенов	200, 401
GET /users/me	Личный кабинет	200, 401
PATCH /users/me	Редактирование профиля	200, 400, 401
GET /users/me/favorites	Сохранённые рецепты	200, 401
GET /users/me/feed	Лента подписок	200, 401
GET /users	Поиск кулинаров	200
GET /users/{id}	Публичный профиль	200, 404
GET /users/{id}/recipes	Публикации кулинара	200, 404
POST /users/{id}/follow	Подписаться	204, 400, 401, 404, 409
DELETE /users/{id}/follow	Отписаться	204, 401, 404
GET /users/{id}/followers	Подписчики	200, 404
GET /users/{id}/following	Подписки	200, 404
GET /recipes	Поиск с фильтрацией	200, 400
POST /recipes	Создание рецепта	201, 400, 401, 404
GET /recipes/{id}	Страница рецепта	200, 404
PATCH /recipes/{id}	Изменение рецепта	200, 400, 401, 403, 404
DELETE /recipes/{id}	Удаление рецепта	204, 401, 403, 404
POST /recipes/{id}/publish	Публикация	200, 401, 403, 404, 422
GET, PUT /recipes/{id}/ingredients	Состав рецепта	200, 400, 401, 403, 404
GET, PUT /recipes/{id}/steps	Шаги приготовления	200, 400, 401, 403, 404
GET, POST /recipes/{id}/media	Галерея	200, 201, 400, 401, 403, 404
DELETE /recipes/{id}/media/{mediaId}	Удаление медиа	204, 401, 403, 404
POST, DELETE /recipes/{id}/like	Лайк	200, 401, 404
POST, DELETE /recipes/{id}/favorite	Избранное	200, 401, 404
GET, POST /recipes/{id}/comments	Комментарии	200, 201, 400, 401, 404
PATCH, DELETE /comments/{id}	Правка и удаление комментария	200, 204, 400, 401, 403, 404
GET, POST /categories, /cuisines, /tags, /ingredients	Справочники	200, 201, 400, 401, 403, 409

Метод и путь	Назначение	Коды ответов
GET /health	Проверка состояния	200

9. Ключевой эндпоинт витрины

GET /recipes реализует требования FR-6 и FR-7. Фильтры комбинируются по «И»; параметры ingredientId и tag допускают повторение, и рецепт должен содержать все перечисленные значения. Без авторизации возвращаются только опубликованные рецепты.

```
parameters:
  - { name: q, in: query, schema: { type: string, maxLength: 160 } }
  - { name: categoryId, in: query, schema: { type: integer, format: int64 } }
  - { name: cuisineId, in: query, schema: { type: integer, format: int64 } }
  - { name: difficulty, in: query, schema: { $ref: '#/components/schemas/Difficulty' } }
  - { name: maxCookTime, in: query, schema: { type: integer, minimum: 0 } }
  - name: ingredientId
    in: query
    style: form
    explode: true
    schema: { type: array, items: { type: integer, format: int64 } }
  - name: sort
    in: query
    schema:
      type: string
      enum: [newest, oldest, popular, cookTimeAsc, cookTimeDesc]
      default: newest
```

Листинг 2 — параметры фильтрации GET /recipes

10. Пример запроса и ответа

Создание рецепта передаёт вместе с основными полями состав и шаги, чтобы клиенту не приходилось делать три последовательных запроса.

```
POST /api/v1/recipes
Authorization: Bearer <access_token>
Content-Type: application/json

{
  "title": "Оссо буко по-милански",
  "summary": "Томлёная телячья голяшка с белым вином.",
  "categoryId": 3,
  "cuisineId": 1,
  "difficulty": "hard",
  "prepTimeMinutes": 30,
  "cookTimeMinutes": 120,
  "servings": 4,
  "tags": ["ужин", "изысканное"],
  "ingredients": [
    { "ingredientId": 1, "quantity": 1200, "unit": "г", "note": "нарезать поперёк" }
  ],
  "steps": [
    { "stepNumber": 1, "instruction": "Обжарить мясо до корочки.", "durationMinutes": 12 }
  ]
}
```

Листинг 3 — запрос на создание рецепта

```
HTTP/1.1 201 Created
Content-Type: application/json

{
  "id": 12,
  "title": "Оссо буко по-милански",
  "slug": "osso-buko-po-milanski",
  "status": "draft",
  "difficulty": "hard",
  "cookTimeMinutes": 120,
  "servings": 4,
  "likesCount": 0,
  "commentsCount": 0,
  "author": { "id": 1, "username": "dr_lecter", "fullName": "Hannibal Lecter" },
  "ingredients": [ { "id": 31, "ingredient": { "id": 1, "name": "Телячья голяшка" },
    "quantity": 1200, "unit": "г", "note": "нарезать поперёк" } ],
  "steps": [ { "id": 47, "stepNumber": 1, "instruction": "Обжарить мясо до корочки.",
    "durationMinutes": 12 } ],
  "isLiked": false,
  "isFavorite": false,
  "createdAt": "2026-04-01T10:15:00.000Z"
}
```

Листинг 4 — ответ на создание рецепта

11. Модель безопасности

Используется схема `bearerAuth` (JWT). Токен доступа живёт час, `refresh`-токен — неделю. Публичные операции (поиск рецептов, просмотр профилей и справочников) объявлены с пустым требованием `security`, что в спецификации выражено как `security: []`. Операции над справочниками, изменяющие их состав, требуют роли администратора.

12. Проверка спецификации

Спецификация проверена валидатором `swagger-parser`; на этапе проверки была найдена и исправлена ошибка: в поточной записи YAML описание с запятой разрывало объект схемы `TokenPair`. Итоговый результат:

```
$ npm run validate:openapi
[openapi] спецификация валидна: Lecter's Recipes API 1.0.0
[openapi] путей: 28, операций: 44, схем: 32
```

Листинг 5 — результат валидации спецификации

13. Выводы

Сформулированы 16 функциональных и 10 нефункциональных требований, обоснован выбор REST как формата API, спроектированы 44 операции на 28 путях с описанием тел запросов, ответов и ошибок.

Спецификация валидна и используется дальше двумя способами: как контракт при реализации ЛР1 и как источник документации, публикуемой приложением в Swagger UI.